

Named Entity Recognition with LSTM Model+N-Grams

Antonio Gagliardi

Master Artificial Intelligence and Robotics

Sapienza, University of Rome

gagliardi.1977681@studenti.uniroma1.it

Abstract—Named Entity Recognition is a popular task in NLP.

Basically we've got a training set full of sentences. Each sentence is composed by a sequence of N words, $\vec{x} = (x_1, x_2, \dots, x_n)$, each one with an attached label $\vec{y} = (y_1, y_2, \dots, y_m)$. These labels indicates the category to which a given word belongs (13 in our task).

Words combined with category define the concept of entity.

Note: It could happens that more than one word contribute to create a single entity. In this case their combination is called chunk.

Index Terms—Entity, Chunk, Sliding Context Window (swc)

I. NER AND BIO-TAGS NOTATION

An entity, as already mentioned, can be made up of either one or more than one word.

The BIO-Tags notation is used to distinguish as "B-", the word that corresponds to the beginning of an entity and "I-", the one or more words that follow. Note that, a word classified as "B-" can exist alone, while a word classified "I-" must always be placed next to a start tag, it cannot stands on its own. The O tag is used to indicate any other word that doesn't correspond to an entity.

The main idea behind the implementation is to prepare a data pack built from many sentences to be supplied as input to the neural network.

Overall ideas there's the main concept that our neural network model is not able to read over string sentences, data need to provided in a numeric representation. All we do is to create a dictionary that maps each single words over a unique number.

Clarified this point, question is how we can retrieve the sentences that will create our dictionary?

To do this, I have come up with 2 different ways of looking at the problem:

1 - We collect the sentences from our dataset and we create the input tensors by performing a sequential parsing on each single sentence by applying a "sliding context window" to each word.

2 - We collect the phrases from our dataset and, by parsing each one, we separate the $Chunks_O$ from the $Chunks_{Entity}$.

II. SLIDING CONTEXT WINDOW MECHANISM

One key features is the "Sliding Context Window": our neural network must learn to label that the concept of

entity is not only linked to a single word but that an entity can also be composed of several words. To do this we must ensure that the network sees as many combination of words as possible, so that it can best level the weights associated with the words within the embedding matrix.

A. N-Grams creation:

Following the first way of working, the combinations are extracted starting from the various input sentences thanks to the *swc*: We define a context length, the algorithm performs a sequential parsing of each sentence and for each word (pivot) extract a context window whose final size will be given by $cw_{length} = 2 * cw_{size} + 1$, where 1 is the pivot element.

Here is an example of result given by my context window extraction algorithm with the following parameters:

- $cw_{size} = 2$
- $stride = 0$

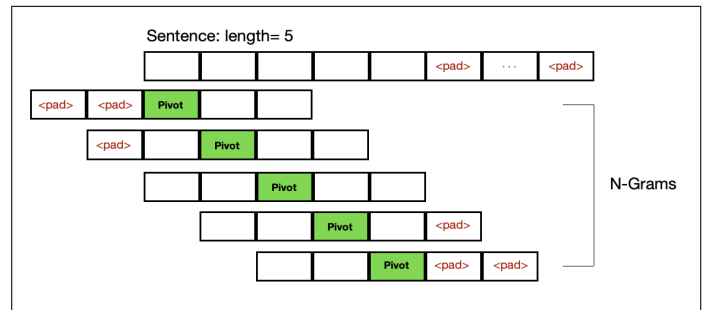


Fig. 1. N-Grams creation: $\langle \text{pad} \rangle$ element are used to uniform the length of the tensors, the pivot can be considered as the actual word of which we want to extract a context

The result is that we obtained what we call *N-Grams* (in a similar way of what we did in "WordVec Bonus Exercise").

Note that our algorithm will reject all the sentences in which the number of padding elements is greater than cw_{size} , because they're useless for our purpose.

B. What's happened during the training:

The confusion matrix can be interpreted in the following way: at the beginning of the training, the weights w_{ij} inside the embedding matrix of the NN are randomly initialized (as the NN was born for its first time). At the end of one training epoch, the network appears to be

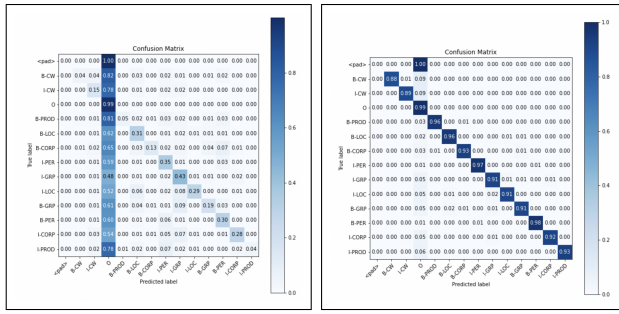


Fig. 2. Confusion-Matrix during the training phase: a)Start of the training process, after 1 epoch of training b)End of the training process after some epoch of training.

very confused, we can retrieve this information visually looking at how the forecasts are related to the target. After some epochs of training, the confusion matrix aligns with values as much as possible close to 1 on the main diagonal, indicating that the network has effectively learnt something and managed to classify our set more or less correctly. The rest of the matrix remains on values as much as possible close to 0, indicating that it makes as little confusion as possible between what is the predicted value for a given word is and what its target value is.

C. Loss and Accuracy trends during the train:

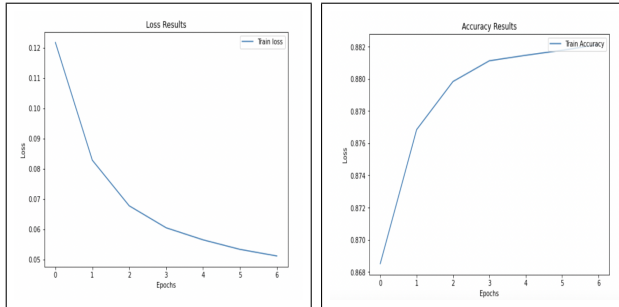


Fig. 3. a) Loss trend during the training phase: from the shape of the curve it seems that the training is proceeding in the right direction, with an adequate learning rate . b)Accuracy goes up along the epochs, this is good, even if it's not the best metric to evaluate this task.

D. Result of the experiments

These are some results of an experiment conducted on my *notebook.ipynb* based on "Sliding Context Window Mechanism" combined with LSTM NN Model with the following configuration:

- Embedding dimension : 512
- Hidden Layer dimension : 256
- Learning rate : 0.01

Dev-Set Classification			
	Accuracy	$F1_{macro}$	$F1_{micro}$
$cw_{size} = 1$	87.3868	0.489	0.336
$cw_{size} = 2$	87.8225	0.499	0.322
$cw_{size} = 3$	87.8213	0.504	0.309
$cw_{size} = 4$	87.7717	0.498	0.295

III. CHUNK RETRIEVING METODOLOGY

The other experimented working methodology aim to process the dataset in an intelligent way before applying the *scw*:

The method consists in directly extracting the chunks corresponding to the various entities from the sentences, making the assumption that a sequence of "O-Tags" is considered a single *Chunk_O* before it is interrupted by a "B-" tag.

Acting this way we're able to retrieve pieces of data on which apply the usual *scw*. This gives us the possibility to treat separately chunks entities and what are not entities. Initial goal of this working method was a try to provide the cleanest possible input data for the neural network without loosing too much words in the dictionary.

The problem with this approach is that the network is not very capable of generalizing, focusing directly on the words that make up a given chunk, but not considering within a chunk entity, for example, a possible sequence of "O-Tags" that precede the starting "B-" tag.

The result is a performance degradation both in terms of accuracy (which is in any case a poorly suited metric for evaluating this task) and f1-score.

IV. CONCLUSIONS

The quantitative results of the problem show a much higher level of accuracy than F1, and this is very likely due to an imbalance of the dataset on the "O" symbols compared to all the other entity symbols. (During training I didn't want to remove O symbols in order not to reduce the vocabulary of the network, but, perhaps it was not the best idea).

In general, I have found that the results obtained through a testing carried out on the notebook (with an evaluation logic similar to the docker) are very different, I have not yet been able to fully understand the reason, I hope to be able to discuss it with those who will correct my paper.

REFERENCES

- [1] NLP's course slide : Deep Learning.
- [2] NLP's course slide : POS Tagging.
- [3] NLP's course notebooks.